BDT PROJECT - A.Y. 2025/26

Campaign Finance & Donor Network Transparency Explorer

A serverless medallion-architecture data lake that analyzes U.S. federal campaign contributions at scale to surface donor concentration, recurring donor networks, and unusual spending patterns.

Team 9

- **Ilva Abazaj** (265350) — BSc Sociology [*ilva.abazaj@studenti.unitn.it*]
- **Lucrezia Pagotto** (263632) — BSc Digital Management [*lucrezia.pagotto@studenti.unitn.it*]
- **Nicholas Perini** (263915) — BSc Sociology [*nicholas.perini@studenti.unitn.it*]

At a glance

- **Source material**: FEC bulk data + OpenFEC API
- **Data processing**: medallion-architecture using MinIO and DuckDB
- **Interactive dashboard**: graphs and filters to investigate how money moves for federal elections cycles and how each candidate gets financed, using Streamlit

TEAM

Team, Roles, And Collaboration

Who worked on what, how the team collaborated, and each member's background

- **Ilva Abazaj** (BSc Sociology): took care of the data exploration, of writing the common mapping document for the Silver Layer and preparing the presentation document
- **Lucrezia Pagotto** (BSc Digital Management): took care of developing the code for the project and organize the team's work
- **Nicholas Perini** (BSc Sociology): took care of developing the code for the project

How we interacted - channels used

GITHUB: to share the code and work on the same versions of the project

Google Drive: to work together on the presentation and share minor documentation

Google Meet: to discuss our ideas live

CONTEXT

Project At A Glance

A compact summary of our project

- **The source:** US campaign finance data is published by the Federal Election Commission (FEC) and contains millions of data about candidates, committees, and how money flows in, out, and between them during campaigns
- **The Project**: our system bridges together static data and incrementally updated records from the OpenFEC API
- **Target users:** journalists, researchers, curious citizens, other candidates and committees
- **Input data:** 5 FEC bulk files from the 2026 election cycle and live OpenFEC API
- **Main result:** a Gold analytics layer displayed through an interactive Streamlit dashboard exposing donor networks, spending anomalies & money flows

Quick facts

- **Scope:** U.S. federal committees, candidates & contributions
- **Pipeline:** ingest → standardize → analyze → serve
- **Scale & Storage:** ~5.5 GB raw text records optimized to high-performance local Parquet datasets

PROBLEM

Problem And Motivation

Why this project matters and what it is trying to solve

- **Problem**: US campaign finance data is public but fragmented, hard to interrogate and manage with millions of records per file. Current data doesn't get uploaded into static files until the election cycle is over and API data offer inconsistent field names and formats that hinder a comparative analysis using both static and API data.
- **Why it matters**: money flows in politics are a core transparency & accountability issue
- **Goal**: a unified, deduplicated, query-ready dataset that exposes donor concentration and money flows and can be used hussle-free by non-data people
- **Scope**: US federal elections for President, House or Senate

Example use case

A political journalist is interested in knowing how much money a candidate received from committees and individuals and which committees supported them directly. With our dashboard this information is readily available and updated.

SCALE

Why This Is A Big Data Problem

The scale and complexity behind the architecture choice

- **Volume**: 30.5 million records across 5 static files, 10,000 raw records gathered from the API in just 4 calls
- **Variety**: Unifies 5 distinct heterogeneous sources, unheaded, pipe-delimited raw text files with deeply nested REST API JSON payloads
- **Velocity**: FEC campaign data is filed continuously by thousands of active PACs and the data is updated to the API nightly. The system pulls the latest 2026-cycle records from the OpenFEC API on demand, merging them with the static bulk seed in the Silver layer removing duplicates.
- **Why the architecture helps**: Avoids the overhead of traditional relational database clusters. By storing compressed, schema-enforced Parquet files in MinIO and querying them with DuckDB, the pipeline achieves fast analytical processing locally.

Size and Key Bottlenecks Solved

- Bulk files size: ~ 5.5 GB
- Key bottleneck: API pagination and rate limits solved by building a network-resilient engine with safe exponential retry cycles and page caps to prevent API rate-limit lockouts

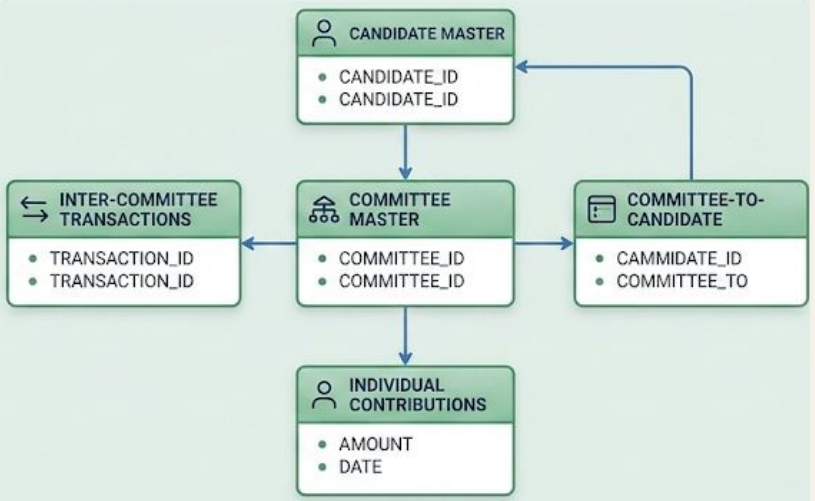
DATA

Data Sources, Quality, And Scope

The data used in this project

- **Data sources:** files from the FEC bulk data section (Committee master, Candidate master, Individual contributions, Inter-committee transactions, Committee-to-candidate contributions)
- **Format Evolution:** FEC bulk (.txt files with .csv headers) + OpenFEC API JSON ⟶ Parquet
- **Real Data Quality Issues Handled:** inconsistent field names, mixed date formats, duplicate transactions
- **Preprocessing Steps:** raw bulk files and live API records are unified into a single schema, field names standardized, dates normalized to ISO YYYY-MM-DD, and duplicates removed - producing clean, query-ready Parquet files in the Silver layer.

What the files contain

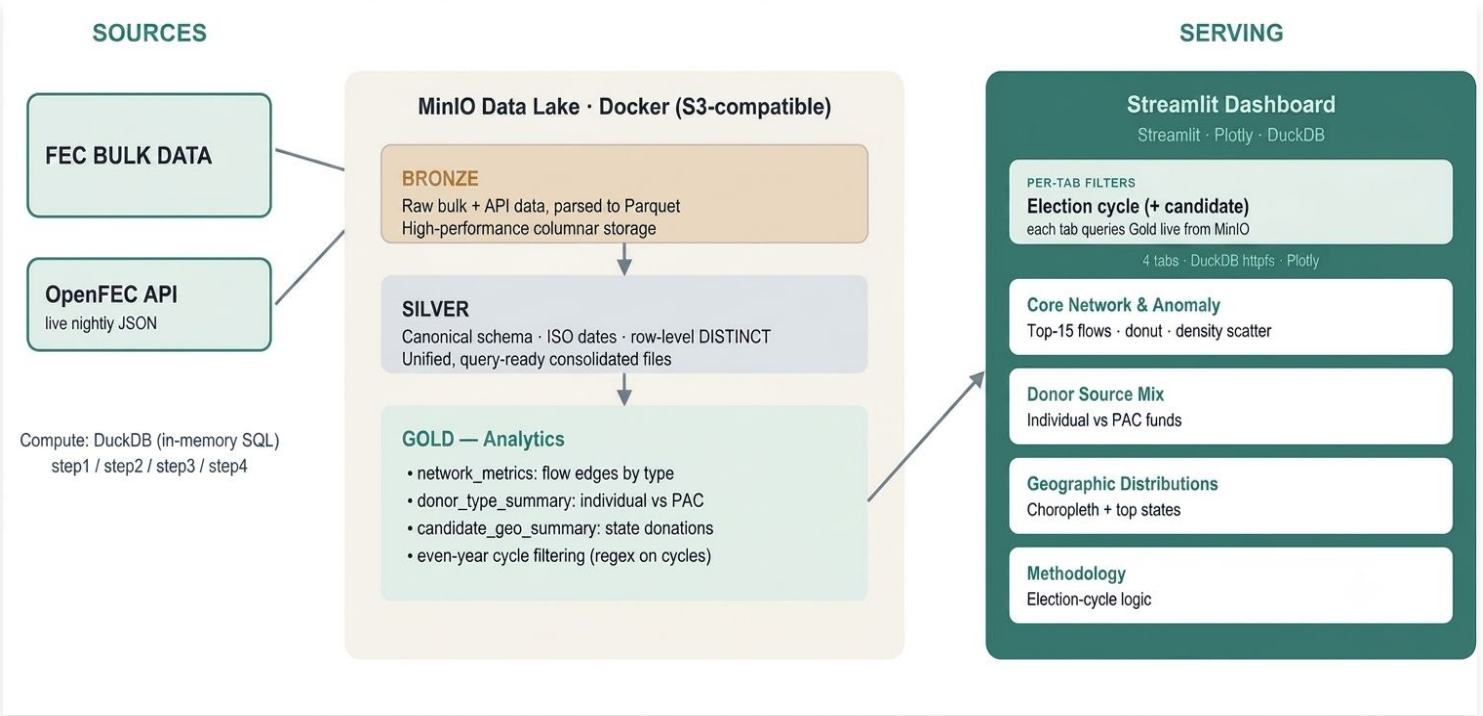


Architecture Overview

Technologies used

- **Sources**: FEC bulk + OpenFEC API
- **Storage**: MinIO (S3-compatible) in Docker
- **Layers**: Bronze ⟶ Silver ⟶ Gold
- **Processing**: Python (steps 1–4)
- **Serving**: Streamlit dashboard

System architecture



PIPELINE

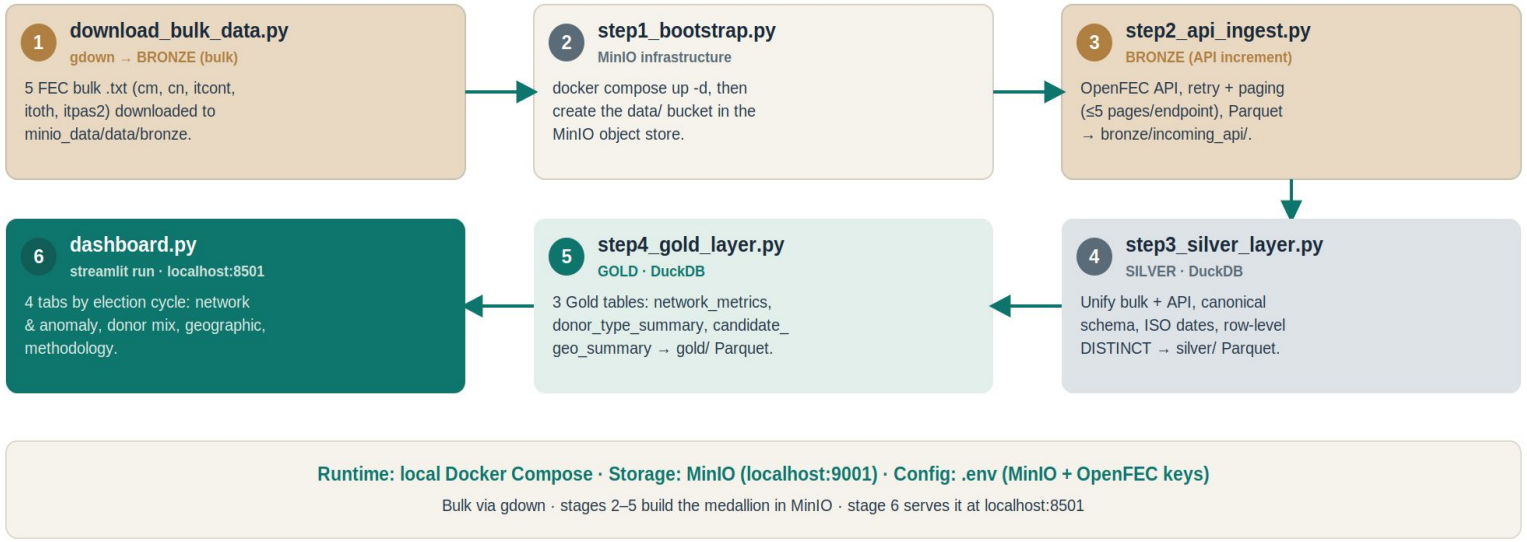
Data Flow, Runtime, And Deployment View

How data moves from ingestion to output

- **Ingestion**: bulk seed + nightly updated API
- **Order**:

1. download_bulk_data.py
2. **step1_bootstrap.py**
3. **step2_api_ingest.py**
4. step3_silver_layer.py
5. step4_gold_layer.py
6. dashboard.py

Pipeline & deployment flow

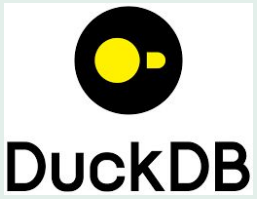


CHOICES

Design Decisions And Trade-offs

Design Decisions

- **Trade-off:** single-node approach combining **DuckDB** and **Parquet** to eliminate network coordination latency, configuration complexity, and heavy infrastructure costs
- **Main technologies and why they are chosen:**
  - **MinIO Object Storage:** Orchestrated via Docker to serve as an S3-compatible local Data Lake.
  - **Parquet Storage Format:** Implemented a columnar format with Snappy compression to replace raw text payloads.
  - **DuckDB Analytical Engine:** streams data out-of-core directly from Parquet files using vectorized execution, allowing high-throughput aggregations without exhausting system memory.
  - **Medallion Data Architecture:** Enforces a strict separation of concerns across a multi-tier data pipeline:.
  - **Streamlit & Plotly BI Layer:** Renders dynamic bar, donut, scatter, and choropleth visualizations. Heavy computations are pushed back into the Gold layer, ensuring the interactive dashboard UI remains responsive and fluid.



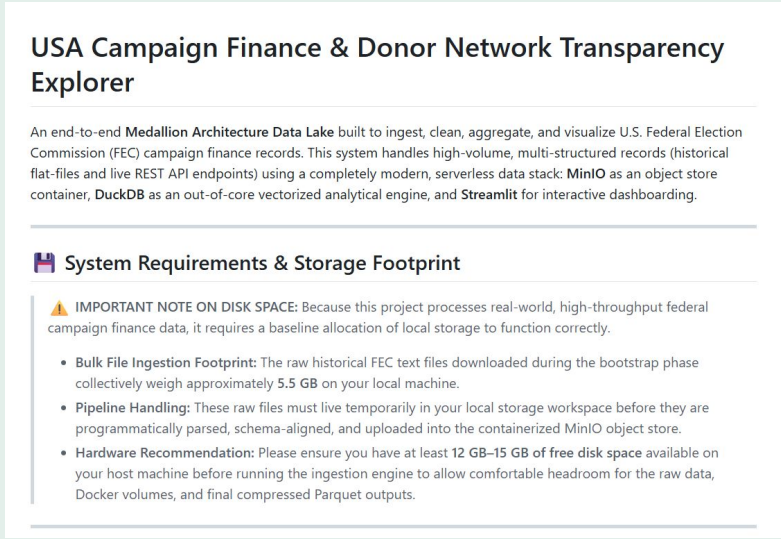
REPRO

Reproducibility And Repository

Where the code lives and how to run the project

- **Github Repo**: Campaign Finance Explorer
- **Prerequisites**:
  - Docker Desktop
  - Python 3.11+–3.13
  - .env file with MinIO and OpenFEC keys (to request here)
  - a virtual environment
  - at least 12-15 GB of free disk space
- **How to run**: clone the github repository in the ***main branch*** and follow the steps described in the README in the same branch
- **The bulk data download needs to happen only once**: if you want to download more data from the API start the pipeline from step2_api_ingest.py and follow the same steps as before. If you simply want to open the dashboard *after* you ran the entire pipeline during previous days, setup Docker and run the command to open the dashboard

Use the README as a guide



USA Campaign Finance & Donor Network Transparency Explorer

An end-to-end **Medallion Architecture Data Lake** built to ingest, clean, aggregate, and visualize U.S. Federal Election Commission (FEC) campaign finance records. This system handles high-volume, multi-structured records (historical flat-files and live REST API endpoints) using a completely modern, serverless data stack: **MinIO** as an object store container, **DuckDB** as an out-of-core vectorized analytical engine, and **Streamlit** for interactive dashboarding.

💾 System Requirements & Storage Footprint

⚠️ **IMPORTANT NOTE ON DISK SPACE:** Because this project processes real-world, high-throughput federal campaign finance data, it requires a baseline allocation of local storage to function correctly.

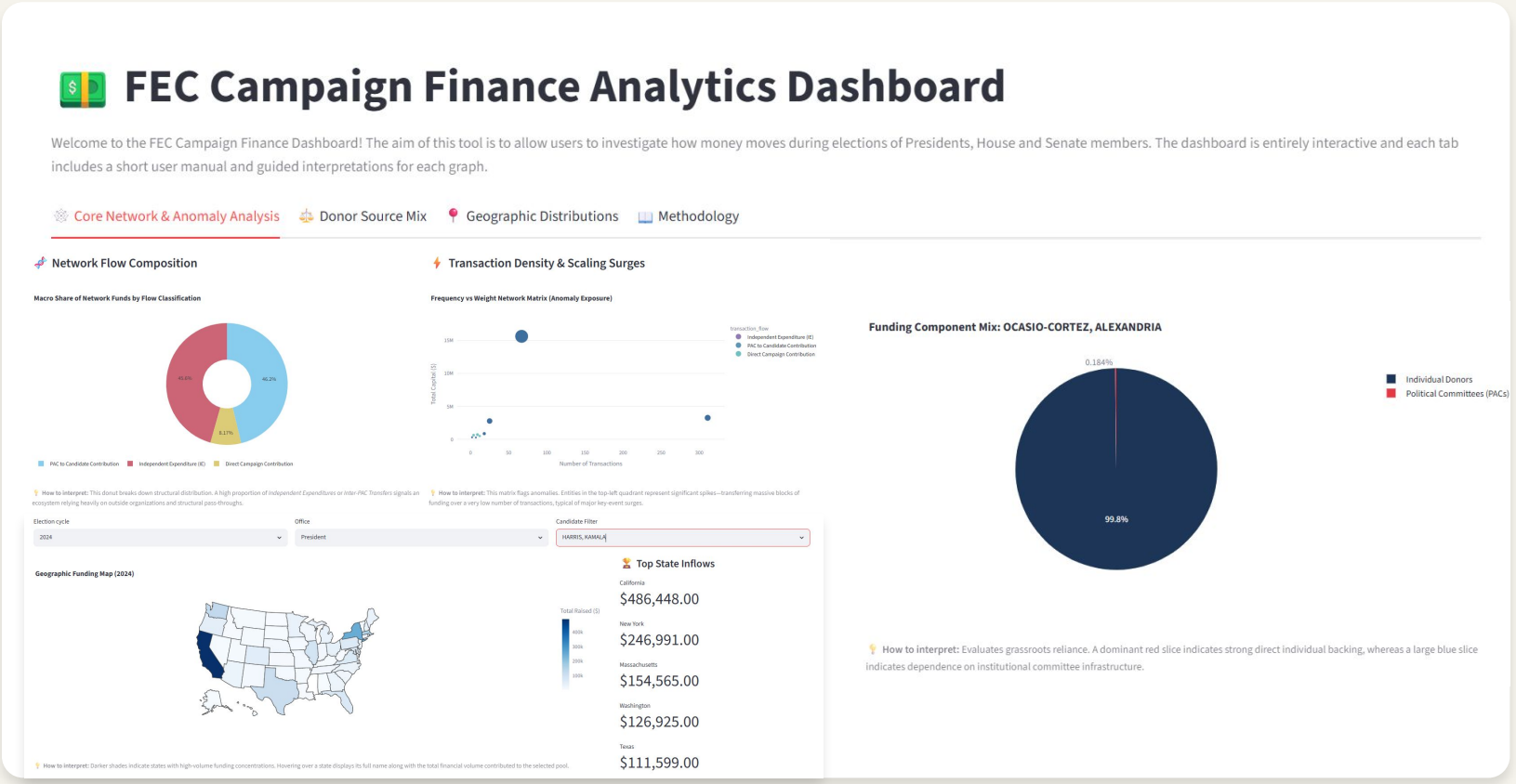
- **Bulk File Ingestion Footprint:** The raw historical FEC text files downloaded during the bootstrap phase collectively weigh approximately **5.5 GB** on your local machine.
- **Pipeline Handling:** These raw files must live temporarily in your local storage workspace before they are programmatically parsed, schema-aligned, and uploaded into the containerized MinIO object store.
- **Hardware Recommendation:** Please ensure you have at least **12 GB–15 GB of free disk space** available on your host machine before running the ingestion engine to allow comfortable headroom for the raw data, Docker volumes, and final compressed Parquet outputs.

Results: Functional Evidence

Dashboard Tabs

- **Core Network and Anomaly Analysis**: to investigate networks between candidates and committees
- **Donor Mix**: to see where a candidate gets their funds
- **Geographic distributions** of individual donors

Visualizations can be filtered for candidates, offices and election cycles



Results: System And Performance Evidence

Runtime and system behavior

Our pipeline delivers exceptionally fast execution speeds by utilizing DuckDB's vectorized execution engine, which processes millions of rows in just a matter of seconds rather than minutes. Furthermore, the system maintains a highly efficient, lightweight RAM footprint by using streaming and compressed Parquet chunks, proving it can scale out-of-core without requiring expensive or heavy hardware infrastructure.

Performance area

Step	Runtime	Peak RAM Consumption
step1_bootstrap.py	0.1 seconds	32.25 MB
step2_api_ingest.py	89.80 seconds	103.55 MB
step3_silver_layer.py	129.05 seconds	965.07 MB
step4_gold_layer.py	35.02 seconds	688.65 MB

Note: the runtime for *download_bulk_data.py* is approximately 7 to 8 minutes: however, this only needs to be ran once after cloning the directory

REFLECTION

Limitations, Lessons, And Next Steps

Reflections

- **Current limitations**: runs at local scale (single-machine Docker); coverage limited to ingested cycles. The Silver layer re-processes all bulk text entries alongside new-real API files during every execution cycle. While this strategy ensures data integrity and drops the risk of duplicated records, it wastes compute cycles on static data as the volume scales
- **Next steps**:

  1.**Source Partitioning**: separate the bronze data layer into distinct directories (e.g.,/source=bulk_static/ vs /source=api_delta/) to skip processing unchanging historical files.

  2.**Incremental table merges**: replace standard parquet files with an ACID table format like Delta Lake or Apache iceberg, using 'MERGE INTO (UPSET) operations to process new records.

  3.**Incremental Gold Views**: set up automated data tools to refresh calculations only for the specific candidates modified by the newest API entries.

Optional additions

- **Scaling concern**: move from local MinIO to distributed/cloud storage to store static files
- **Stretch goal**: real-time anomaly alerts near key electoral dates. Transitioning from redundant local batch processing to an incremental, cloud-scalable Lakehouse architecture.

INTEGRITY

References, Acknowledgments, And GenAI Disclosure

Sources, external help, and acknowledgments.

- **Datasets & Infrastructure:** Official FEC Bulk Data Portals, the OpenFEC REST API, and MinIO Object Storage systems.
- **Core Engineering Stack:** DuckDB, PyArrow, Snappy Parquet Compression, Streamlit, and Plotly Graphing Suites.
- **GenAI Tool Usage Disclosure:** Gemini was used as an AI project collaborator to help design data structures, optimize SQL queries for data cleanup, align disparate schemas across data layers, and refine the final presentation architecture. All code was tested, validated, and run locally. Claude was used to counter-check statements and coherence of documents with the scripts.

Source Links

- OpenFEC API
- Bulk download files from fec.gov